BUY SOMETHING
Toade3

CUSTOMER APPLICATION | TECHNICAL BLUEPRINT

Customer Mobile App
End to End Architecture

Toade3 Campus Logistics Platform

Native customer experience with a Supabase backend

Identity, ordering, tracking and verified delivery

DOCUMENT ID

ARCH-TOADE3-CUSTOMER-2026-V1

VERSION

1.0.0 draft

18 October 2026 | Engineering and product review

Mobile application architecture | Ghana campus operations | Confidential

Document guide

This document defines the end-to-end architecture of the Toade3 Customer mobile application. It connects the customer screens to authentication, catalog data, pricing, order transactions, live tracking, delivery proof and operational services. It is a proposed build blueprint, not a report of an implemented system.

Control	Baseline
Document and version	ARCH-TOADE3-CUSTOMER-2026-V1 | 1.0.0 draft | 18 October 2026
Related documents	Toade3 TRD v1.1.0 and architecture.md v1.0.0 created in this project
User application	Installed Customer mobile app; Android and iOS are proposed release targets
Shared platform	Supabase Auth, PostgreSQL, Realtime and Edge Functions
Design system	Toade3 navy #152244, orange #E8682A and light orange #FDEFE6; original logo retained
Scope boundary	Vendor and Rider apps are external participants. Payments, settlement and refunds remain outside the current baseline.

Contents

1 System context	3	11 Offline and recovery	13
2 Customer app internals	4	12 Customer handover proof	14
3 Screens and navigation	5	13 Data and access architecture	15
4 Identity and sessions	6	14 Customer API contracts	16
5 Catalog cart and drop points	7	15 Security privacy and permissions	17
6 Quote and pricing	8	16 Deployment and release	18
7 Order placement and retry safety	9	17 Performance and operations	19
8 Order lifecycle and cancellation	10	18 End to end validation	20
9 Tracking and foreground updates	11	19 Delivery plan and decisions	21
10 Push and app lifecycle	12	Appendix Sources and design baseline	22

Confirmed: mobile delivery and inherited business rules. Proposed: framework/library choices, read API names, support floors and operating thresholds. Review items are collected on page 21.

1 System context

The Customer app owns the customer experience. Supabase owns authoritative identity and business data. Vendor preparation and rider delivery change shared order state through their own authorized commands; customers receive a limited view of those changes.

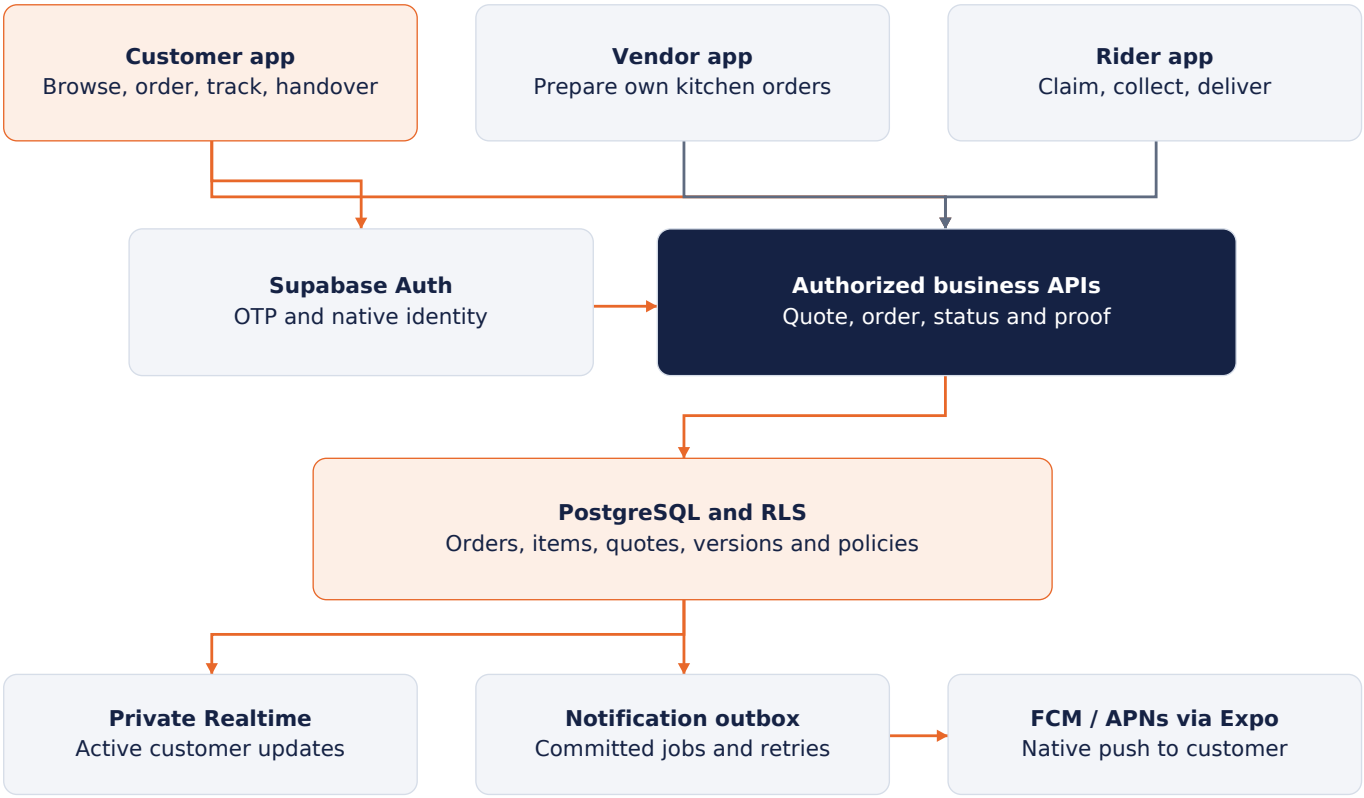


Figure 1. Shared platform context. Realtime and push return to the Customer app; neither channel can create business state on its own.

Trust and ownership

Boundary	Rule
Device to backend	The client sends intent and identifiers. The server verifies actor, ownership, prices, weight and state.
Customer to operations	The customer can read own orders and cancel eligible ones; cannot prepare, claim or complete delivery directly.
Tracking and proof	Only active own-order tracking is exposed. The customer holds the secret; only the assigned rider submits it for server verification.

Customer-only device scope
The baseline Customer app uses notifications, secure storage and local caches. Foreground location is optional for selecting a drop point. Camera scanning and continuous rider background tracking belong to the Rider app.

2 Customer app internals

A React Native Customer app is the proposed implementation, using Expo development builds and TypeScript. Native stacks and tabs connect screens to application services; server data, local drafts, secrets and device permissions have separate owners.

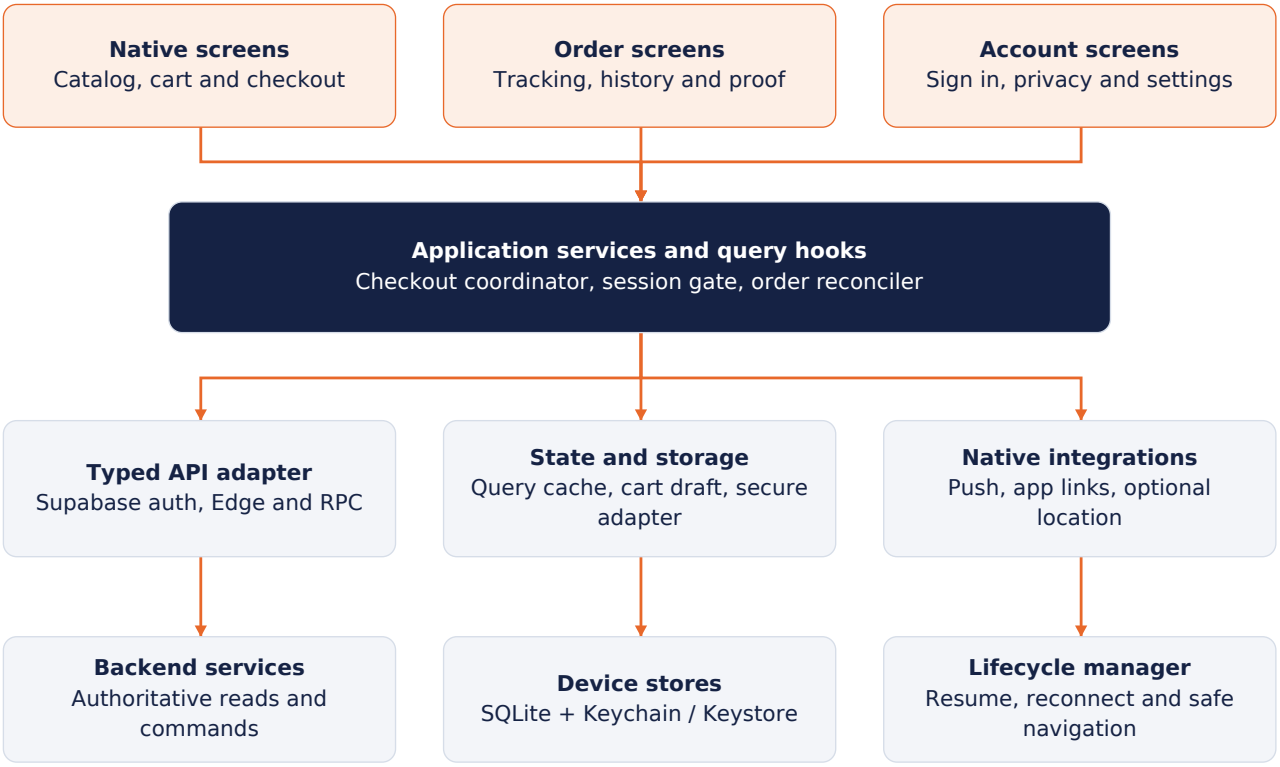


Figure 2. Customer app containers. Shared domain types and validators serve the services; backend enforcement remains independent of client code.

Layer	Implementation responsibility
Presentation	Accessible native controls; loading, pending, empty, stale and error states; no direct business-table writes.
Application state	TanStack Query for server snapshots; cart/UI state in component state or Zustand; no duplicated authoritative order state.
Persistence	SQLite for bounded, versioned read caches and drafts. An encrypted OS-backed adapter stores tokens and customer proof credentials.
Integration	Typed DTOs, stable errors, idempotent commands, private subscriptions, push registration and AppState recovery.

Module placement

apps/customer contains the native entry point and screens. Shared packages/ui, auth, api, domain, realtime, storage and device provide reusable components and contracts. Do not import Rider-only camera or background-location workflows into Customer screens.

3 Screens and navigation

The primary navigation uses Home, Orders and Account tabs. Cart and checkout are guarded screens, and order detail / tracking / handover are addressed by an authorized order id. Screen names below are proposed native navigation identifiers.

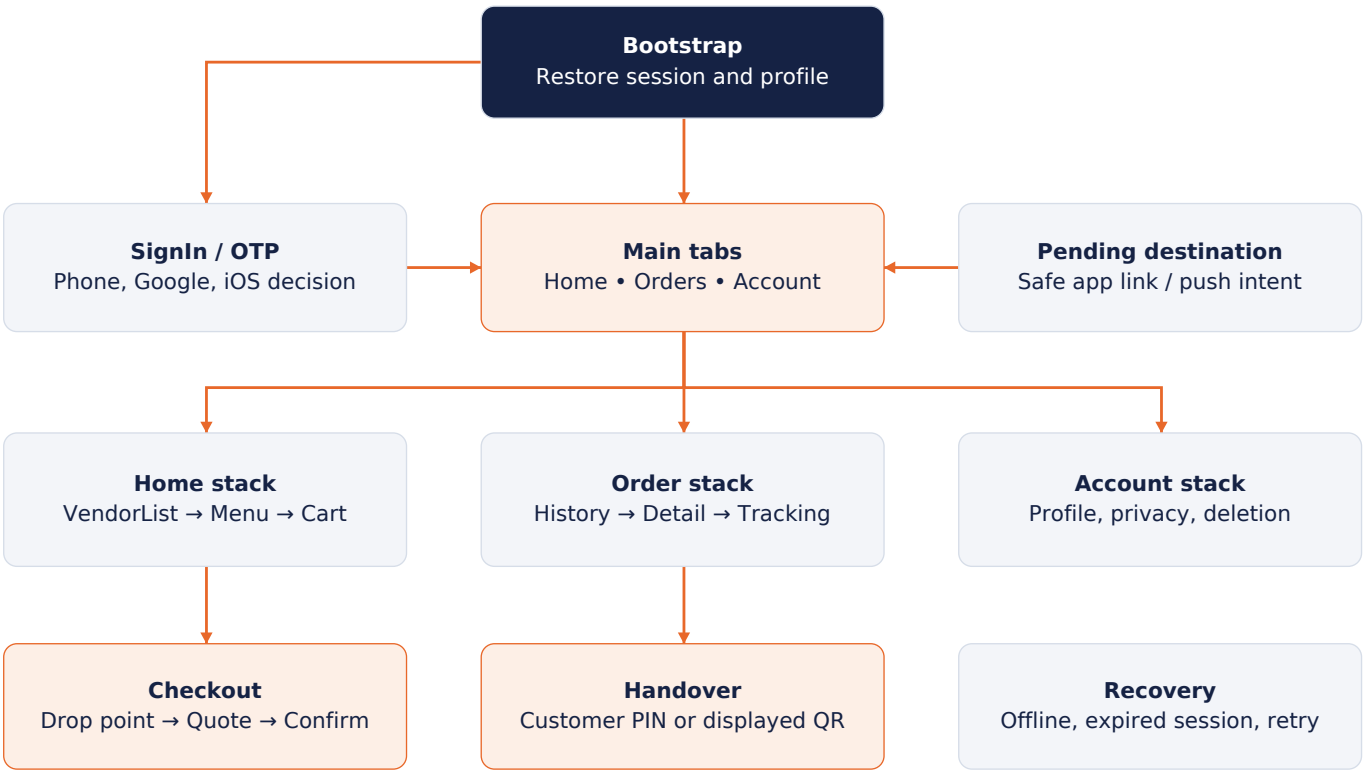


Figure 3. Navigation relationships. Pending external destinations wait until bootstrap succeeds; recovery is a state handled by affected screens, not a way around authorization.

Navigation contract

- Bootstrap holds the splash screen until secure session restoration and profile loading resolve. Missing session opens SignIn; a wrong role opens a role mismatch screen.
- A deep link or notification supplies an intent, not authorization. Validate allowed destinations and re-read the resource before opening it.
- Sign out or account switch resets the root stack, subscription manager and per-user caches so back navigation cannot reveal another session.
- A delivered or cancelled order remains readable in history with an appropriate terminal state; its active rider tracking and proof actions are removed.

Core customer journey
Sign in → choose vendor and items → select campus drop point → review quote → confirm order → follow preparation and delivery → present PIN or QR → view completed receipt.

4 Identity and sessions

Use Ghana phone OTP and native Google sign in through Supabase Auth. Every new identity receives a customer profile. The mobile role guard and server checks both require the customer role for customer commands.

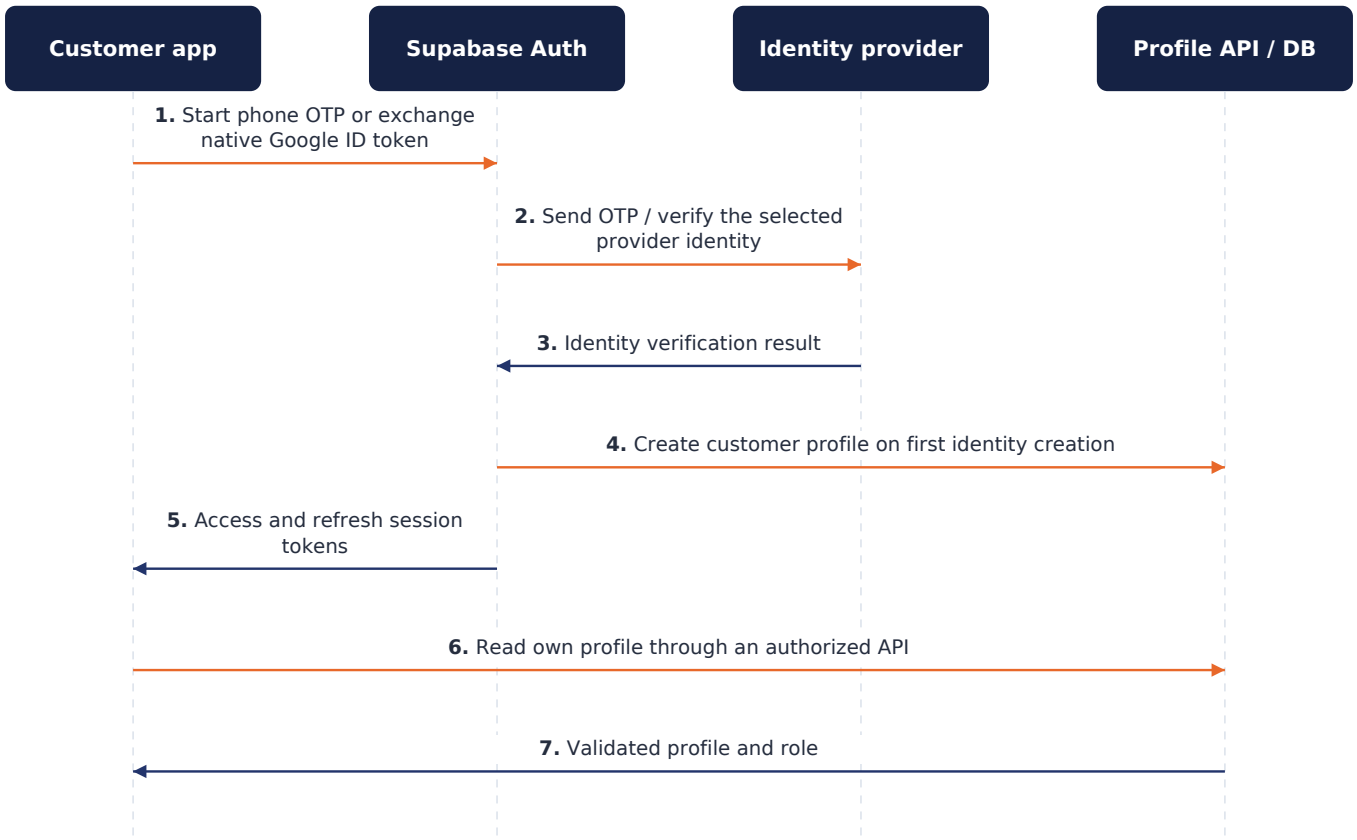


Figure 4. Logical identity sequence. Phone code entry / verification and Google token acquisition follow their provider-specific steps before the session is accepted.

Concern	Customer requirement
OTP controls	Normalize +233 numbers; numeric code entry; supported OTP autofill without SMS-inbox permission. Server limits, resend timing and spend caps follow the TRD.
Session storage	Encrypted native adapter; token-size handling, refresh rotation and concurrent refresh protection. Never store tokens in plain SQLite or logs.
Lifecycle	Refresh while active; restore and recheck profile on resume. Do not assume suspended timers run. No Customer background GPS task is needed.
Callbacks and roles	Validate allowlisted callbacks, state / PKCE where used, cold launch and cancel paths. Do not assign roles or merge identities from unverified client metadata.
iOS release	Resolve Apple guideline 4.8 for primary Google login; proposed equivalent Sign in with Apple where required, unless a valid exception applies.

Session failure behavior
Keep cart intent where safe, return to sign in when needed, and refetch current order state after reauthentication. Never replay a new checkout simply because a session refresh completed.

5 Catalog cart and drop points

The customer selects items from one vendor and a controlled campus drop point. Local drafts make browsing responsive; an authorized server read and fresh quote determine what can actually be ordered.

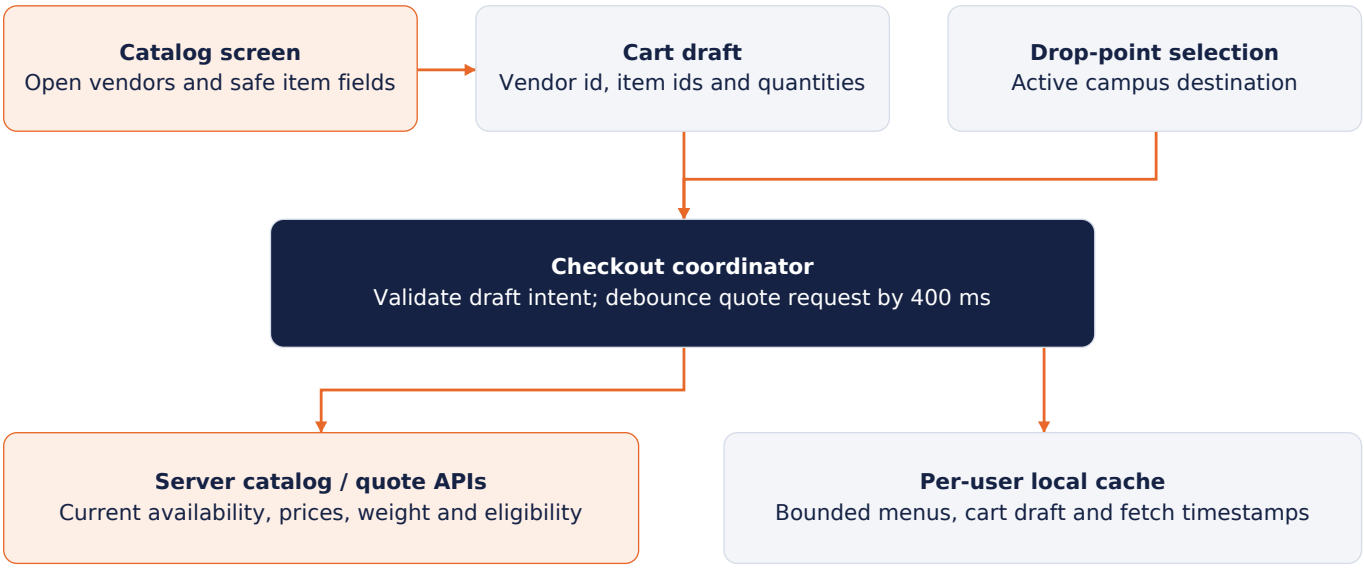


Figure 5. Catalog-to-checkout data flow. Quotes use server data; cached menu prices and stock are presentation only.

Read and draft contracts

Concern	Behavior
Catalog projection	Return vendor identity/name/opening state and public-safe menu fields. Exclude owner contact data and internal fee splits.
Cart invariant	One vendor per order. Quantity must be positive and every item must belong to that vendor. A vendor change requires an explicit cart replacement decision.
Availability	Unavailable items cannot be quoted or placed. Stock changes invalidate menus; cached availability carries a last-updated label.
Destination	Use an active drop_point_id. Optional foreground location suggests a destination; it never replaces server-approved coordinates.
Cancellation of reads	Debounce rapid draft edits, cancel obsolete requests where supported, and display only the newest matching quote.
Sign-in boundary	Public-safe catalog reads may be allowed by backend policy. Quote and placement require an authenticated customer; local cache does not create a session.

Customer-visible states

Loading skeleton; empty menu; vendor closed; stale offline menu; unavailable item; missing drop point; quote pending; valid quote; quote expired. Keep these states explicit so an old fee cannot be mistaken for a confirmed checkout total.

No checkout from a stale draft
When connectivity returns, reload current item eligibility and obtain a new quote. Preserve quantities where valid, explain changes and require customer confirmation of the current total.

6 Quote and pricing

quote-delivery reads current vendor and drop-point coordinates, item prices and weights, then applies the inherited Haversine fare matrix. The server returns an expiring quote bound to the customer and cart fingerprint. Exact storage and expiry duration are implementation decisions.

Zone	Priced distance	Delivery fee GHS	Target time
Inner Quad	0.0 to 1.0 km	5.00	10-15 min
Mid Campus	1.1 to 2.5 km	8.00	15-25 min
Outer Perimeter	2.6 to 4.0 km	12.00	25-35 min
Extended Zone	Over 4.0 to 5.0 km	12.00 + 2.50 × (priced_km - 4.0)	35+ min

Server calculation order

1. Validate customer role, current item/vendor eligibility and active drop point. Read canonical prices and positive known item weights.

2. Calculate vendor-to-drop-point Haversine distance and round upward to 0.1 km. Reject priced distance above 5.0 km.

3. Apply the tier, calculate item total and payload, reject a single order above 8,000 g, and calculate the grand total with decimal money arithmetic.

4. Return quote_id, cart fingerprint, fare version, expiry, distance, zone, delivery_fee, items_total, total_amount and total_weight_g. Placement must validate again inside its transaction.

Illustrative customer checkout

Line	Calculation	Result
Example menu items	2 × GHS 20.00	GHS 40.00
Payload	2 × 800 g	1,600 g
Distance and zone	2.0 km → Mid Campus	Delivery GHS 8.00
Grand total	GHS 40.00 + GHS 8.00	GHS 48.00

Boundary and accounting checks

Raw km	Priced km	Fee GHS	Internal split: commission / rider
1.04	1.1	8.00	1.50 / 6.50
2.55	2.6	12.00	2.00 / 10.00
4.50	4.5	13.25	1.99 / 11.26
5.01	5.1	Rejected	OUT_OF_RADIUS

Visibility and review status
Customers see items, zone/distance, fee and total. Internal commission and rider payout are excluded from customer responses. Tier-gap rounding and the extended charge basis retain the TRD working defaults and require commercial confirmation before release.

7 Order placement and retry safety

An order is created by one authenticated command with a persisted idempotency key. The client sends the quote identifier, not a trusted fee or customer id. A successful acknowledgement represents a committed order, not a payment receipt.

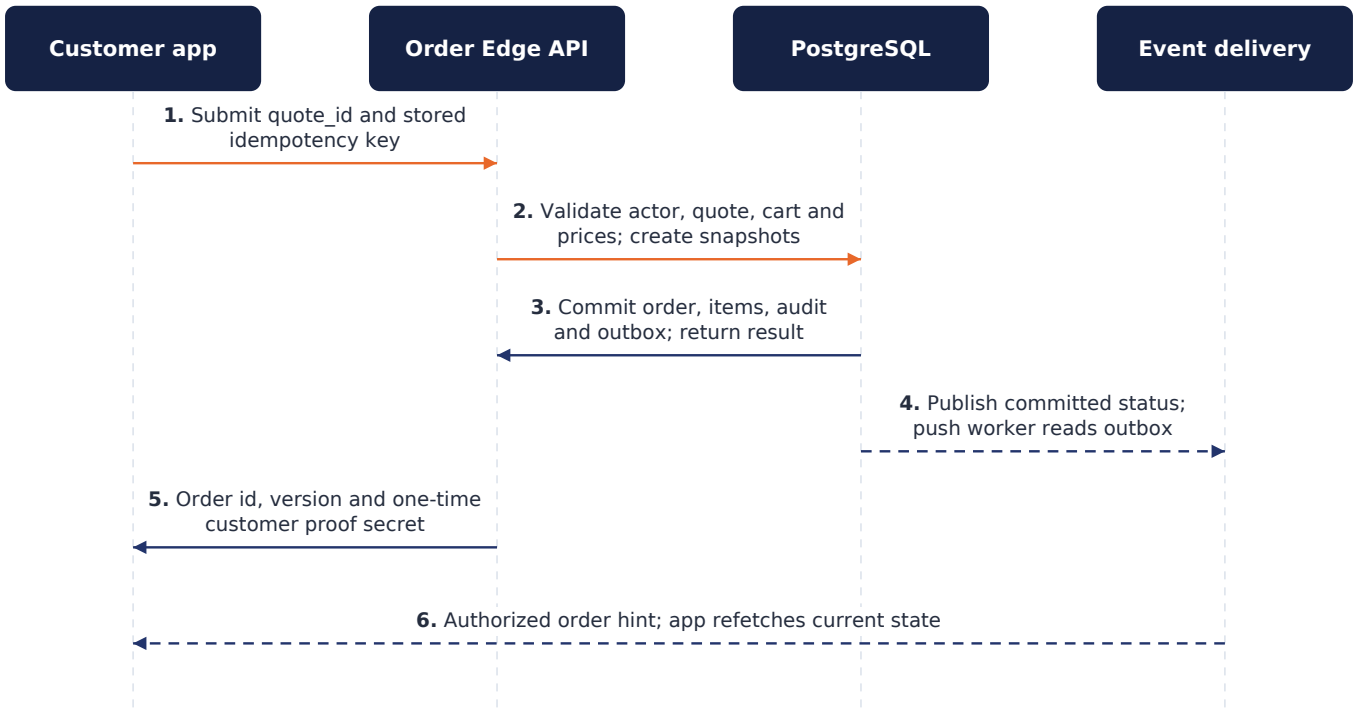


Figure 6. Order placement and post-commit updates. Pricing and snapshots are revalidated in the transaction; event delivery cannot roll back or duplicate the business order.

Transaction contract

Stage	Invariant
Before mutation	Derive actor from verified session. Match quote owner, expiry and cart fingerprint. Reject a reused key with a different request fingerprint.
Atomic mutation	Re-read eligibility and prices, then write the order, immutable items, money split, protected proof verifier, audit and durable outbox together.
After commit	Return authoritative ids and versions. A repeated key resolves to the existing order outcome and cannot create a second order.
Changed quote	Return QUOTE_CHANGED / expiry or availability error; fetch a new quote and require explicit customer confirmation.

Lost response and handover recovery
A replay returns the same business order result, without storing a readable PIN in general idempotency records. If the one-time proof response was lost, reauthenticate and reissue the customer credential through the isolated handover service. Never create another order to recover a PIN.

8 Order lifecycle and cancellation

The customer observes the order lifecycle but does not own kitchen or delivery transitions. The backend publishes an authoritative version after every permitted change. Display a pending action until the relevant command is acknowledged.

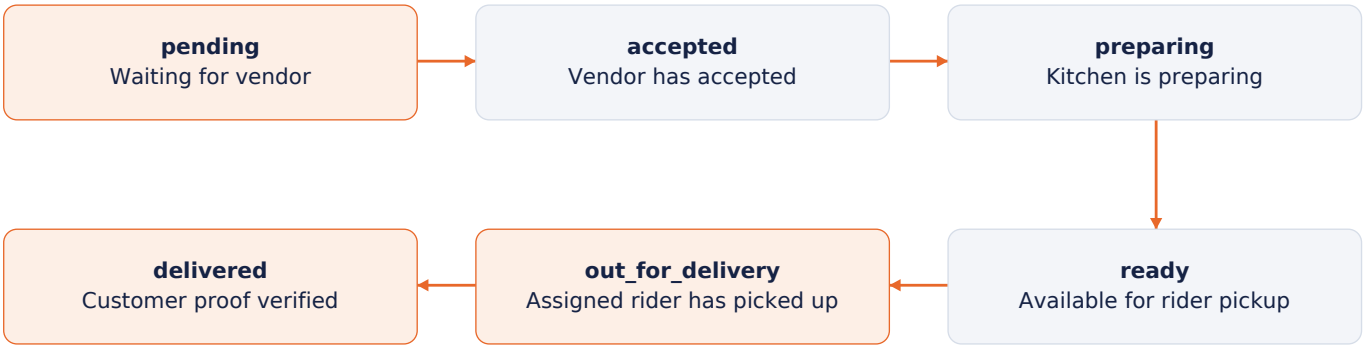


Figure 7. Main order path. Rejected and cancelled are terminal branches governed by the rules below.

Customer status / action	Authoritative actor	Customer behavior
Pending → accepted / rejected	Own vendor	Update status and explanation from current server state; rejection includes a reason.
Accepted → preparing → ready	Own vendor	Show progress. Do not infer preparation from elapsed time or an optimistic local timer.
Ready → out_for_delivery	Assigned rider pickup command	Show assigned rider / ETA and eligible tracking after the server accepts pickup.
Out for delivery → delivered	Handover verification service	Remove active proof and tracking actions; show completed order summary.
Cancel pending / accepted	Order customer via cancel_order	Allowed before preparation. Send expected_version; show pending until the transaction completes.
Cancel races with preparation	Backend locks and version check	Exactly one legal result wins. Refetch on STALE_VERSION; never show cancellation solely from the tapped button.

Cancellation cleanup

Within the same transaction, cancel the order, release any unpicked assignment, update batch eligibility and retain an audit event. All lock-taking paths follow the dispatch lock order so cancellation and claim cannot deadlock or leave a live orphan job.

External dependencies that affect the customer

The Rider service enforces two active orders, an 8.0 kg batch, pickup separation within 300 m and the 5.0 km delivery radius. Customer screens may explain eligibility failures but cannot bypass those rules. Post-pickup exceptions, refunds and settlement remain open operational scope.

Terminal-state behavior
Keep the receipt / history entry accessible under ownership checks. Stop active tracking subscriptions, clear local proof material and reject new handover use once the order is delivered, rejected or cancelled.

9 Tracking and foreground updates

The Tracking screen combines a current order snapshot with private status and assigned-rider events. All reads are scoped to the customer own order. A rider location is a time-stamped observation, not an estimate that can be invented when updates stop.

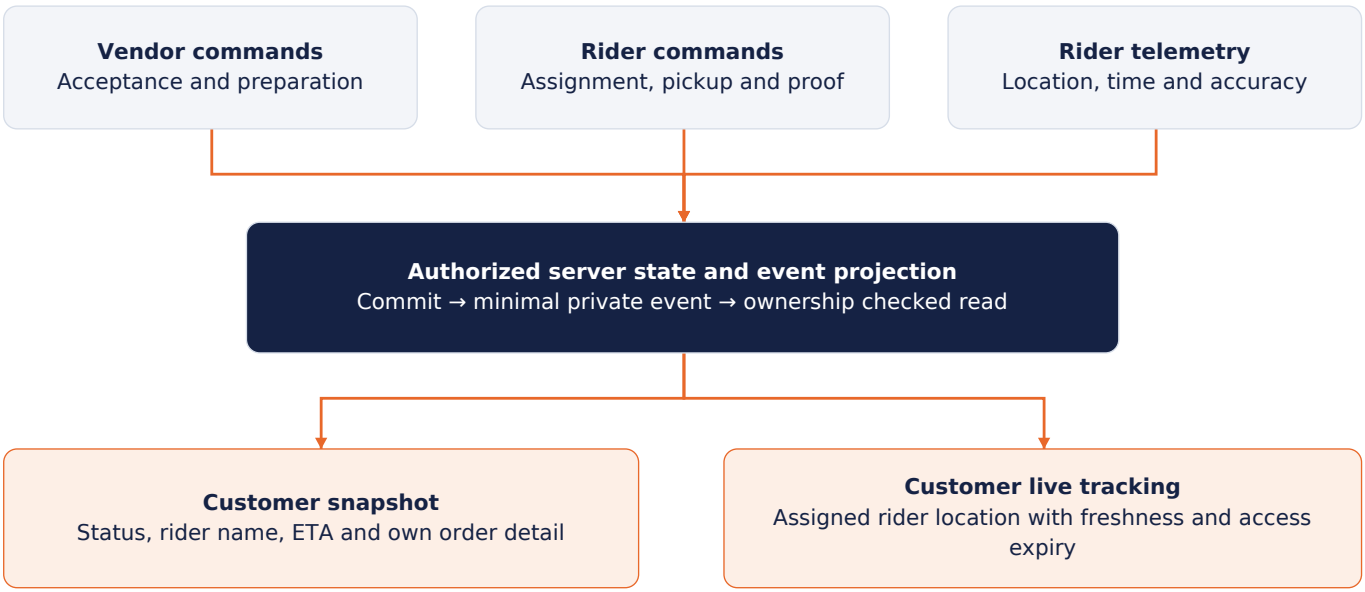


Figure 8. Customer tracking projection. Exact public method names for the tracking read / subscription are proposed contracts, not existing endpoints.

Subscribe and reconcile

1. Authorize the private order topic and buffer incoming events while obtaining a versioned snapshot.
2. Apply only events newer than that snapshot; deduplicate event ids and invalidate affected query keys.
3. Refetch after foreground return, reconnect and notification taps. Use an active reconnect backoff of 1, 2, 4, 8 and at most 15 seconds.
4. Label stale location with last observed time. The proposed rider freshness threshold is 90 seconds; validate it operationally and never display a fake moving marker.

Contract	Rule
Privacy window	Tracking exists only for the active own order and its assigned rider. Terminal state removes topic membership and read access.
Foreground target	Vendor acceptance to active Customer render p95 below 500 ms over 200 trials; no trial above 1,000 ms. This is a target, not a measured result.
Event payload	Resource id, event id and version are sufficient for invalidation. Do not broadcast the full order row, proof verifier, contacts or internal money fields.

10 Push and app lifecycle

Native push alerts bring the customer back to current server state. They complement foreground Realtime but do not replace it. A notification is a hint that something changed; its text or delivery receipt cannot confirm an order transition.

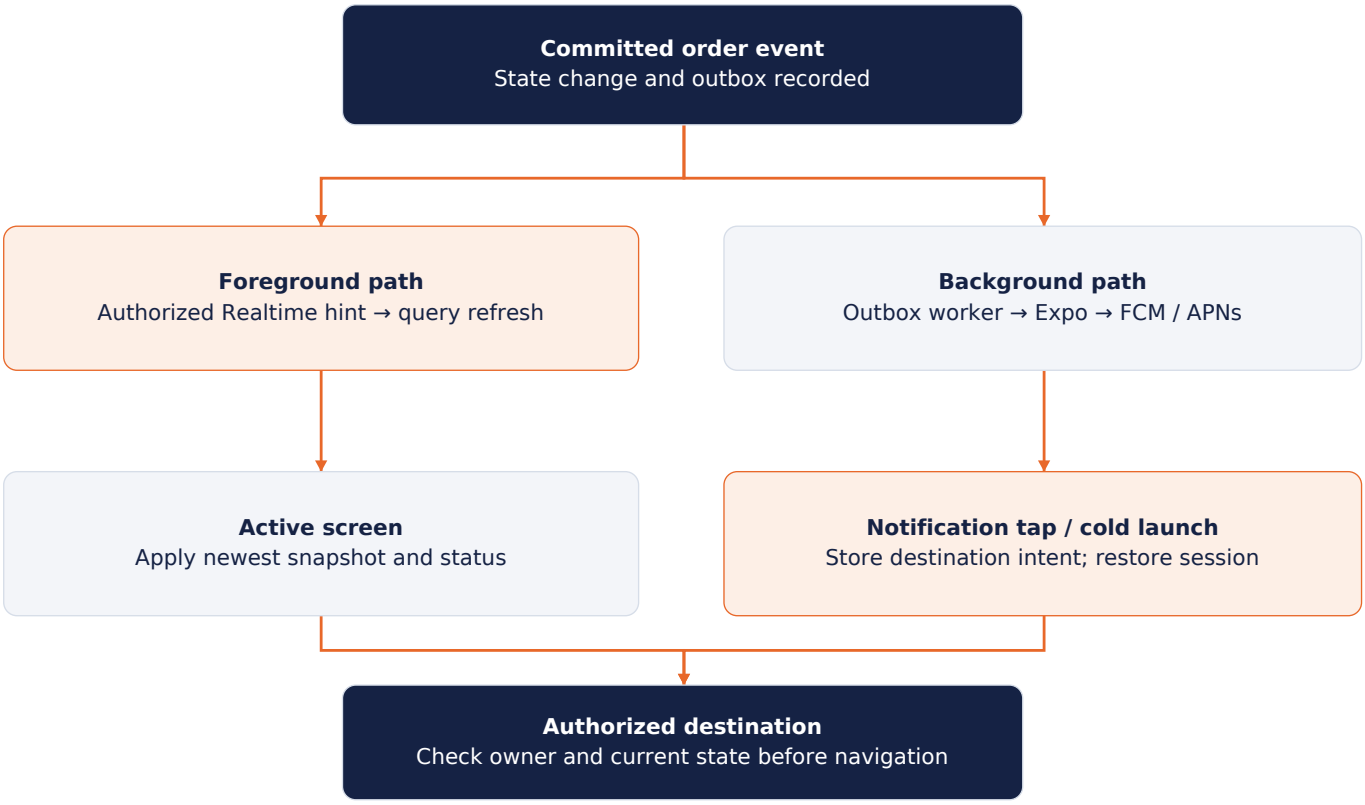


Figure 9. Active and background delivery paths converge on an authorized fresh read. Push delivery timing is outside the foreground 500 ms target.

Lifecycle event	Required action
Register / rotate token	Bind the current installation, customer user, environment and app identity. Update changed tokens; registration is not proof of role.
Enter background	Do not assume the WebSocket or JS timers remain active. Preserve safe pending operation metadata; rely on server state and permitted native notifications.
Resume / notification tap	Restore or refresh session, reload profile and affected order, renew authorized subscriptions, then open an allowed destination.
Duplicate / delayed alert	Deduplicate event ids and read current state. If the order already ended, open history rather than reviving an old live tracking screen.
Sign out / account switch	Revoke the installation association, clear pending destinations and proof secrets, cancel old subscriptions and reset navigation.

Notification privacy
Send a generic message, resource id, event id and version. Exclude PINs, QR tokens, phone numbers, precise rider coordinates and internal money fields from lock-screen payloads. Notification denial leaves the core app usable.

11 Offline and recovery

Offline support is for safe reading and draft preservation. Placing or cancelling an order and verifying delivery require connectivity. The app must distinguish a request that failed before submission from a request whose committed outcome is unknown.

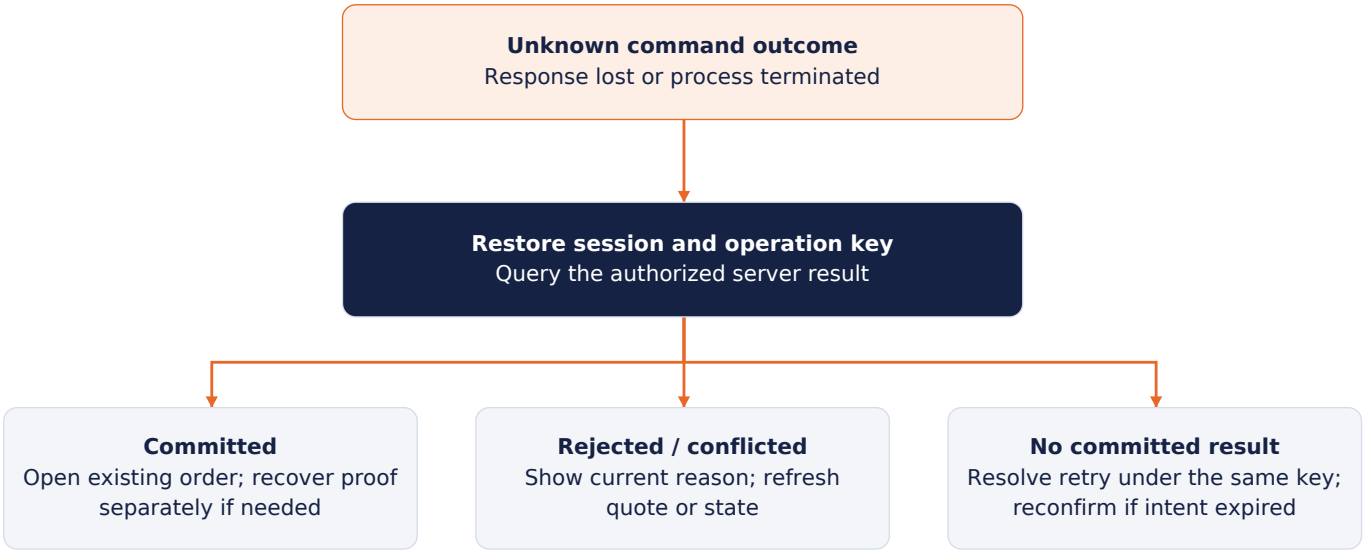


Figure 10. Ambiguous request recovery. A retry uses the original key and fingerprint; a changed cart or new intentional order receives a new key only after reconfirmation.

Local data or action	Storage and recovery policy
Menus and vendor list	SQLite cache with fetched_at and version. Label stale stock and opening state; refresh before checkout.
Cart draft	Per-user vendor/item/quantity intent. Recheck item eligibility, destination and total after reconnect.
Order history / receipts	Bounded redacted read cache. Server history remains authoritative; cached status shows its age.
Session and handover	Encrypted OS-backed adapter. Clear on sign out, account change or terminal order; no proof secret in ordinary SQLite.
Pending operation	Persist opaque operation id, actor scope, idempotency key and request fingerprint before sending. Never store a readable PIN in general replay records.
Write commands	No new automatic offline business-command queue. Resolve ambiguous results first; avoid optimistic completion or duplicate placement.

Recovery states on screen

Show Offline for unavailable connectivity, Confirming order for an ambiguous placement, Reconnecting for live updates, Quote expired for a stale amount, and Sign in required for an expired identity. Keep the action and the uncertain result visible; do not erase the cart and silently restart checkout.

12 Customer handover proof

The customer presents a PIN or displays a signed QR code at handover. The Rider app enters or scans it and submits it to verify-handover. Only the server can move the order to delivered after validating the assigned rider and proof.

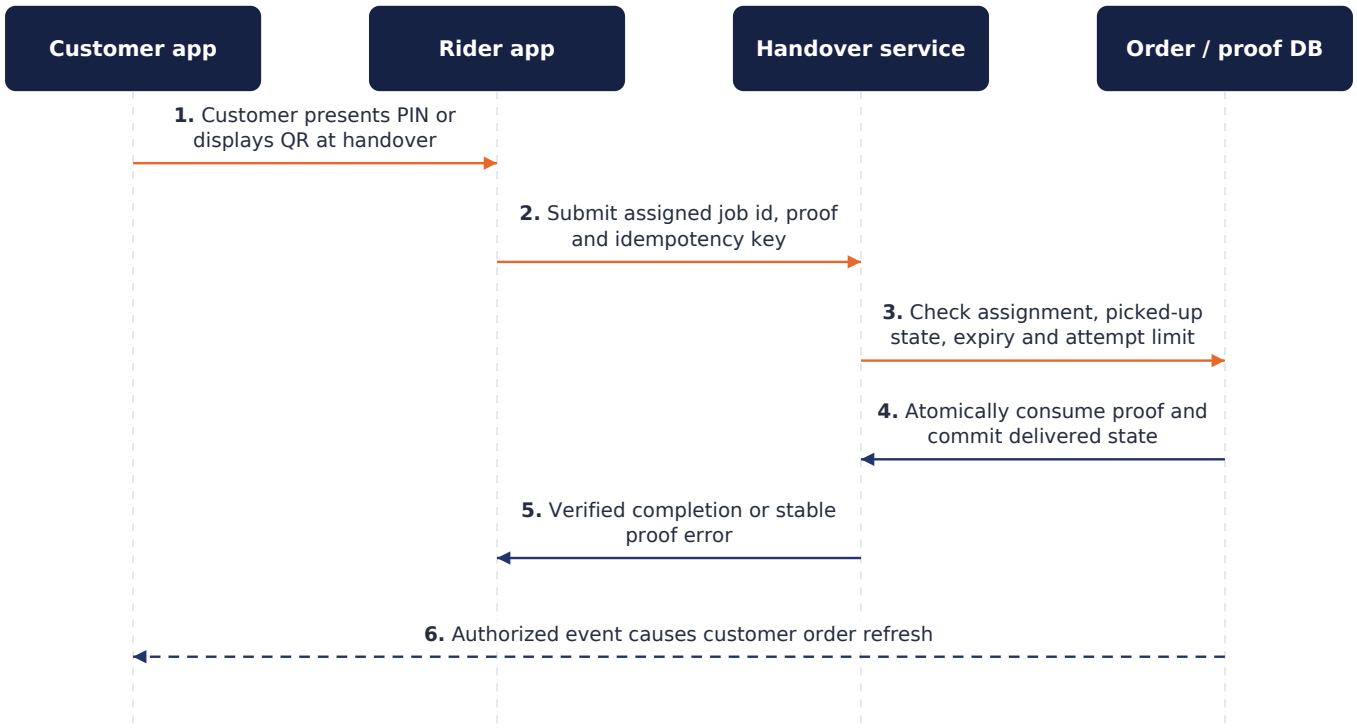


Figure 11. Logical handover flow. The first exchange is the customer presenting proof in person, not a network API between the two apps.

Proof concern	Customer / server contract
PIN issue	Place-order returns the customer credential once. Persist securely; server retains a protected verifier with a server-held secret, never readable proof in a general order response.
QR issue	After pickup, a customer-only authenticated request issues a signed token with order id, nonce and expiry. Working lifetime: 30 minutes from pickup.
Failed attempts	Five incorrect PIN attempts lock verification for 10 minutes and notify operations. Treat expired / reused QR as invalid.
Recovery	Reauthenticate the owner and explicitly reissue lost/expired proof. Invalidate predecessor credentials, rate-limit and audit the action; rider APIs cannot recover customer secrets.
Terminal state	Consume the credential once, remove active tracking access, clear local proof and update customer history. Repeated successful verification resolves to the same completion.

Device permission boundary
The Customer app renders the QR and does not need camera permission for this flow. The Rider app requests scanning permission and offers PIN entry as its fallback. Offline presentation does not establish server-verified completion.

13 Data and access architecture

Store canonical customer and order data in PostgreSQL. Provide narrow customer projections; keep private proof, notification and command records separate. Device persistence is a disposable cache and secret store, not a second order database.

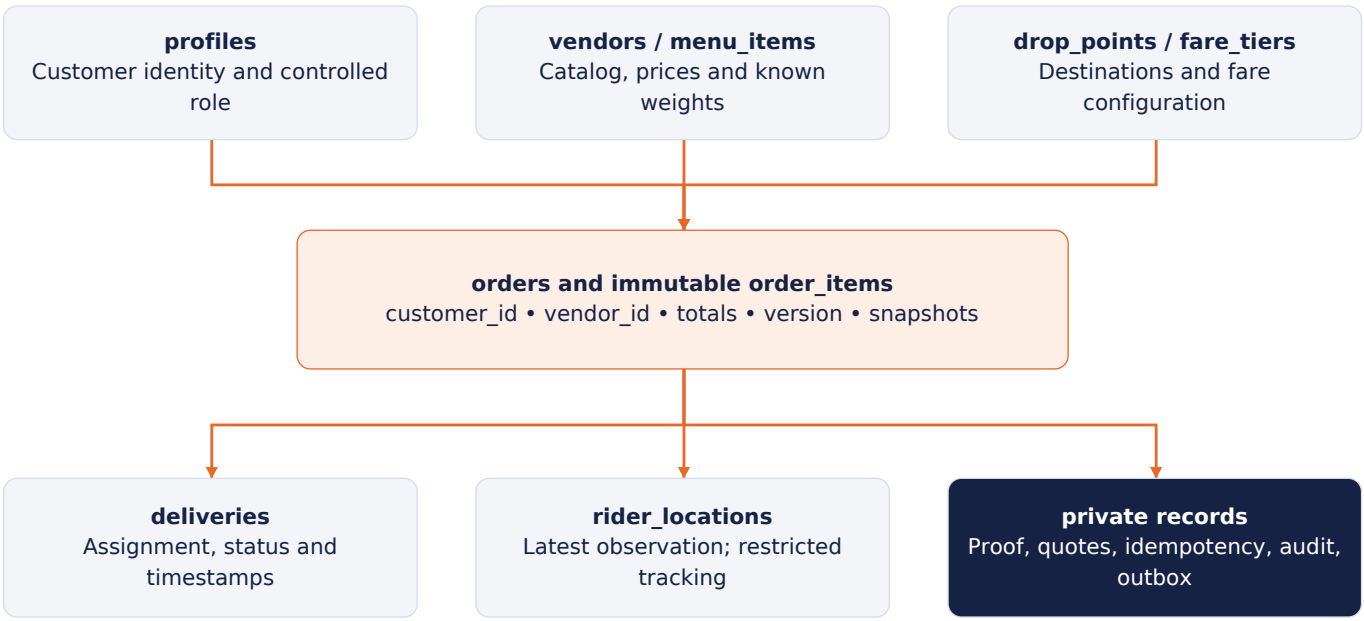


Figure 12. Logical data relationships and projections. There is at most one delivery per order; location is exposed through its active assignment, not a public rider directory.

Resource	Customer access
profiles	Read own identity; update only allowed profile fields. Role / verified contact changes use separate approved workflows.
catalog and drop points	Public-safe vendor/menu projection and active destination list; no owner private details or complete fare commission rows.
orders / order_items	Read own order projection and snapshots; create or change only through validated commands. Do not expose internal commission or proof columns.
deliveries / tracking	Read status for own order and authorized active-rider projection. Remove live location access when the order ends.
devices	Manage own installation through registration endpoints. Validate binding when switching user or environment.
private stores	No direct customer SELECT. Dedicated services validate ownership, quote scope, replay keys and proof recovery.

Campus isolation
The working pilot is one campus with multiple vendors. A multi-campus release requires campus_id and membership checks across orders, catalog, destinations, pricing, device scope and channels, with cross-campus tests before enablement.

14 Customer API contracts

Every customer endpoint verifies session and role where required, derives the actor on the server, validates identifiers and returns typed response objects. Read names marked proposed below complete the customer interface; they are not claims of deployed APIs.

Contract	Purpose and required response
Auth SDK methods	signInWithOtp / verifyOtp and native Google token exchange; session lifecycle in the shared auth adapter.
Catalog read [proposed]	Paged safe vendors/menu data, opening/availability state, price, weight and cache/version metadata.
Destination read [proposed]	Active campus drop points; coordinates and identifiers for quote input.
quote-delivery [Edge]	Vendor, destination and item quantities → quote_id, expiry, fare version, fee, weight and total.
place-order [Edge]	quote_id + Idempotency-Key → order id, version, current totals and one-time proof response on initial issue.
get_order [RPC]	Own order id → role-safe order/items, delivery state and eligible tracking reference.
My orders read [proposed]	Own order history with pagination; excludes proof secrets and internal settlement data.
cancel_order [RPC]	Own order id + expected_version → cancelled state or a stable conflict.
Tracking read / topic [proposed]	Active own order → assigned rider projection and last observed time; membership expires with eligibility.
get_handover [Edge]	Owner-only issue/recovery operation; reauthentication for rotation; no rider access.
register-device / unregister-device	Bind / revoke own customer installation and environment push token.
request-account-deletion [Edge]	Reauthenticated customer request → tracked account-deletion outcome/workflow.

Illustrative quote request

```
POST /functions/v1/quote-delivery
Authorization: Bearer <customer_access_token>
{
  "vendor_id": "<vendor_uuid>",
  "drop_point_id": "<active_drop_point_uuid>",
  "items": [{ "menu_item_id": "<item_uuid>", "qty": 2 }]
}
```

Stable failure handling

401 SESSION_EXPIRED → reauthenticate and reconcile. **403** FORBIDDEN → stop restricted access. **409** ITEM_UNAVAILABLE, QUOTE_CHANGED or STALE_VERSION → refresh and confirm. **422** OUT_OF_RADIUS / OVER_WEIGHT → revise the cart or destination. **429** RATE_LIMITED → show the server retry delay. Never label a timeout as a confirmed rejection.

Do not expose operational commands
Customer code cannot call kitchen transitions, claim_job, rider location writes, verify-handover or admin_set_role. Backend role and ownership checks must enforce this even for a modified app.

15 Security privacy and permissions

The Customer app and its requests are untrusted. Use RLS, explicit grants, role-shaped reads and validated command functions. A publishable Supabase key is public configuration, not a substitute for resource authorization.

Risk	Architecture control
Another customer order / IDOR	Validate owner for every order, tracking and proof resource id; test guessed ids and private topic membership.
Price or role tampering	Re-price and verify stored role on the server. Never use user-editable metadata for authorization or trust submitted customer_id / fee fields.
Leaked session / credential	OS-backed encrypted adapter, backup exclusions, log redaction and sensitive app-switcher previews. No tokens or proof in ordinary SQLite / notification payloads.
Malicious app link / push	Allowlist destinations; validate callback state where relevant; restore identity before navigation and re-read current resource ownership.
Unsafe privileged SQL	Control search_path, validate actor/ownership and narrowly grant EXECUTE. Internal helpers/private stores remain non-exposed.
Stale authorization	Sensitive commands check current account/role state. Refresh-session revocation does not imply immediate access-token expiry.
Transport bypass	Require HTTPS/WSS and normal certificate validation; remove production cleartext and permissive certificate exceptions.

Customer permission matrix

Capability	Baseline rule	Denied path
Notifications	Ask after a clear customer-facing explanation; respect OS settings.	Core screens continue; show settings guidance.
Foreground location	Ask only when Use my location is selected.	Manual campus destination selection.
Camera / background GPS	Not required by this Customer baseline.	Rider scanning and rider telemetry remain external dependencies.
Contacts / microphone / SMS inbox	Not required for ordering, OTP entry or proof display.	Do not request unrelated permissions.

Account lifecycle

Provide an accessible privacy notice and reauthenticated account-deletion request. Operations must approve retention and anonymization rules for orders, audit, device tokens, proof attempts and caches. Resolve active jobs before removing account access; no unchecked cascade should destroy the operational record.

Ownership ends where the purpose ends
A customer sees the assigned rider only for an eligible active order. Completion or cancellation revokes tracking and proof access and clears local proof material. History remains restricted to the customer own order.

16 Deployment and release

The deliverable is a signed Customer app for the selected mobile platforms plus a compatible Supabase backend. Development, staging and production use separate backend configuration, callback identifiers and push registrations.

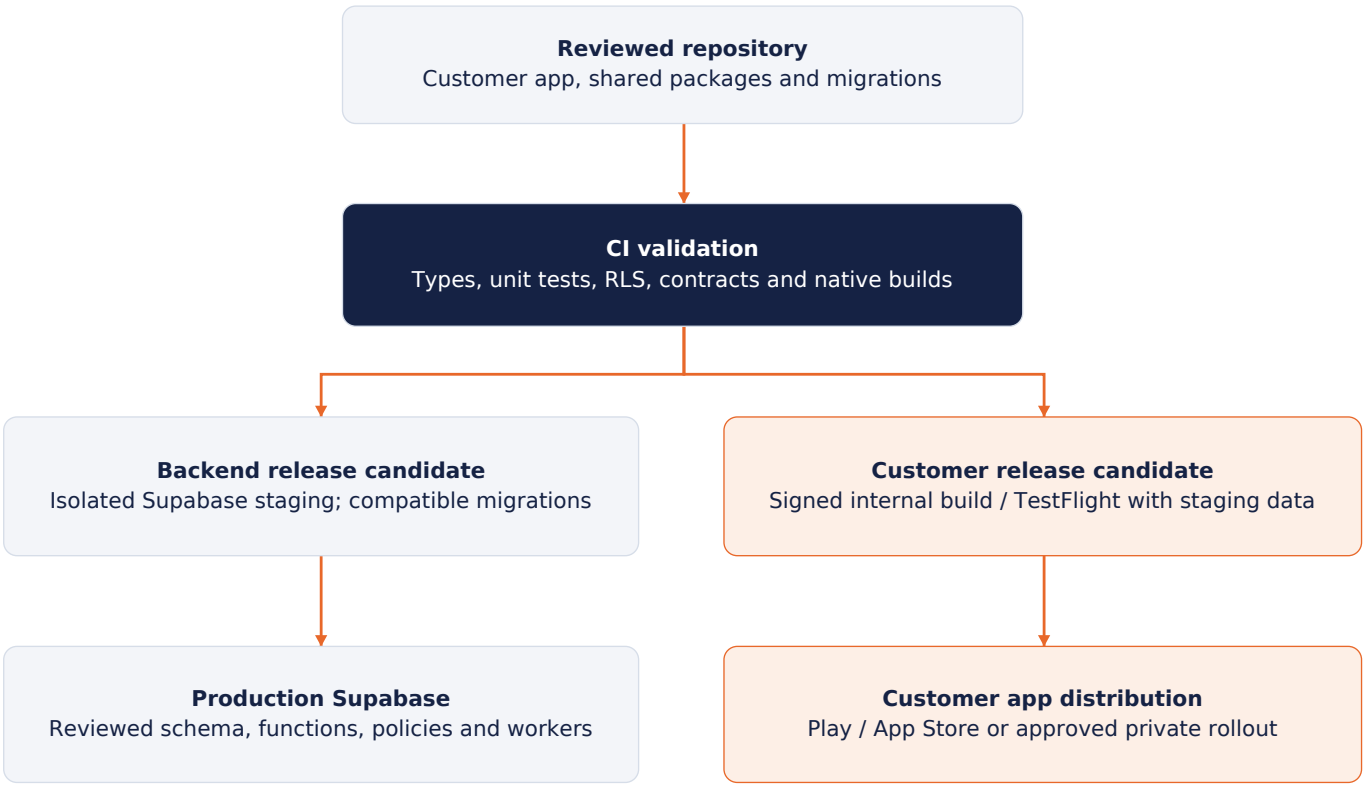


Figure 13. Customer delivery pipeline. Native candidates must pass integration and physical-device checks against the matching backend before controlled production rollout.

Release concern	Requirement
Environment isolation	Separate app ids / callback allowlists, backend projects, push scope and datasets. Never permit an internal staging build to write production by default.
Public vs secret config	App may contain project URL, publishable key and allowed public app configuration. Signing, service/secret keys and push-provider credentials stay in protected server / CI stores.
Store readiness	Organization-owned accounts, signing, provider configuration, privacy / data safety disclosures, deletion access, review instructions and the iOS login decision.
Compatibility	Older installed clients remain supported during rollout. Use expand-and-contract changes, versioned contracts, feature flags and an approved minimum supported app policy.
Recovery	Pause phased rollout, apply a compatible corrective release or approved runtime-compatible code rollback. Native entitlement / SDK changes require a new signed binary.

Active orders during upgrades
An update-required state must preserve a safe route to resolve an active order. Backend rollout precedes dependent native features and cannot assume every customer updates immediately.

17 Performance and operations

These are architecture budgets and sizing assumptions, not achieved performance claims. Validate them with signed Customer builds, real campus network conditions and the selected Supabase region and tier.

Area	Target / assumption	Verification
Foreground status	p95 below 500 ms over 200 vendor-acceptance to customer-render trials; no trial above 1,000 ms.	Common-clock native harness with network/device/region and overhead recorded.
Quote response	p95 below 800 ms on agreed campus data profile.	Authenticated API and native checkout measurement.
Native startup	Usable cached cold screen within 3 s; warm resume within 1 s on reference devices.	Signed release builds with realistic cache/session states.
Rendering	Target 60 fps on reference device; accessible text and controls.	Profile menu/history lists and live map updates.
Payloads	Menu image target at most 60 KB; app-level invalidation event target below 1 KB.	Report transport overhead separately; bound pagination and cache size.
Pilot platform	10,000 registered users, 300 peak orders/hour, 40 riders, 25 vendors.	Load the actual tier and concurrent subscribers; do not infer capacity from user count.
Service availability	99.5% monthly during agreed campus operating hours.	Synthetic ordering/tracking checks and an approved availability definition.

Observability without private payloads

Signal	Use
Customer experience	Startup, screen errors, crashes/ANRs, quote time, order-confirmation uncertainty, reconnects and stale tracking.
Business health	Order placement failures, vendor acknowledgement backlog, rider assignment gaps, proof lockouts and cancellation conflicts.
Platform health	Auth failures / SMS spend, database latency, outbox age, push receipts and invalid-token counts.
Correlation	Carry a request/operation id through API, transaction and events. Exclude JWTs, OTPs, PINs, QR tokens and raw contacts from logs.

Runbooks and recovery

Define owners for elevated order errors, auth abuse, stale tracking and delayed notifications. Provision database backups / point-in-time recovery as required, run restore drills and approve RPO/RTO. Set retention and deletion jobs for caches, proof attempts, device tokens and operational records.

Separate service targets
The 500 ms measure concerns active-app status rendering, not push delivery or food-delivery time. Zone delivery times remain operational targets. The Customer app needs no continuous background GPS for its baseline.

18 End to end validation

Verify the Customer experience against the backend and vendor/rider contracts. Use isolated fixtures during development, then run the final journey with the actual role participants on the selected release platforms. A document or passing unit test alone is not launch evidence.

Gate	Customer scenario	Pass evidence
T1 Identity	Phone OTP / native Google; provider cancel; first and repeat sign in; process restart; sign out.	One customer profile per identity, encrypted session restoration and no prior-user data after switching.
T2 Access	Guess another order id; open protected deep links; subscribe to another topic; try vendor/rider commands.	No restricted screen/data; wrong-role RPC rejected; RLS, grants and topic membership all enforce scope.
T3 Pricing	Run fare vectors and weight/radius boundaries; change stock or price between quote and placement.	Exact GHS totals; no more than 8.0 kg or 5.0 km; revised total requires confirmation.
T4 Realtime	Vendor accepts while Customer tracking is active; reconnect during several transitions.	p95 below 500 ms across 200 recorded trials; no missed final state after recovery.
M1 Permissions	Deny notifications/location, revoke location in Settings; inspect requested permissions.	Manual drop point and core screens work; Customer baseline requests no camera or background tracking.
M2 Notifications	Foreground/background/locked/cold launch; delayed and duplicated alerts; token rotation; sign out.	Generic payload, fresh authorized navigation, no secret exposure and no stale event resurrecting an old job.
M3 Tracking dependency	Interrupt rider reporting, change assignment, complete or cancel order.	Last-known time shown; proposed stale threshold honored; terminal tracking read/topic access removed.
M4 Offline and release	Disconnect before/after commit, kill/relaunch, retry same key, install a compatible upgrade.	One order only; pending operation recovered; lost proof uses isolated recovery; active order remains resolvable.

One complete release journey

1. Sign in → choose a vendor → add items → choose campus drop point → obtain and confirm a current quote.
2. Place order → vendor accepts/prepares/readies → rider claims and picks up → Customer sees current status, assignment and freshness.
3. Customer presents proof → assigned Rider submits → server completes once → Customer history updates and live proof/tracking access closes.
4. Repeat with a cancellation race, lost placement response, expired session, denied permission and invalid proof. Preserve the correct final state in every case.

Release evidence to retain
Customer version/build, backend migration/function revision, device/OS, selected network profile, synthetic dataset, test results and measured targets. Real provider sign in and native notification checks require signed-device smoke tests.

19 Delivery plan and decisions

Implement one coherent customer journey before broadening features. The roadmap below groups dependencies; estimates and staffing belong in a separate delivery plan. This document keeps mobile delivery and the inherited order/fleet rules fixed while marking implementation choices for review.

Increment	Customer deliverable	Exit condition
1 Foundation	Native shell, sign in, session adapter, customer guard, shared DTOs and safe read APIs.	T1/T2 and native callback smoke tests pass.
2 Ordering	Catalog, cart, drop point, quote, atomic placement and pending-operation recovery.	T3 and lost-response / same-key tests pass.
3 Delivery experience	Own-order state, tracking, private subscriptions, push, handover and cancellation.	T4 and complete role-integrated journey pass.
4 Release hardening	Permission fallbacks, privacy/deletion flow, cache lifecycle, accessibility, signed builds and operations.	M1-M4, release evidence and open gates closed.

Decisions to close

ID	Decision	Working direction
C-1	Platforms and packaging	React Native / Expo Customer binary for Android and iOS; confirm launch order, SDK compatibility and proposed Android 10 / iOS 16 support floor.
C-2	iOS identity and ownership	Organization-owned store/signing/provider accounts; add equivalent Apple login where required or document a valid exception.
C-3	Read APIs and event contracts	Finalize catalog/history/tracking read names, schemas, allowed topics and version semantics before integration.
C-4	Map and destination experience	Native compatible map module/provider; campus drop points; optional foreground location. Confirm licensing before offline resources.
C-5	Quote and command lifecycle	Approve quote TTL, fingerprint rules, key/result retention and credential issue/recovery behavior.
C-6	Operational thresholds	Confirm 90-second tracking freshness, vendor escalation, support recovery and measured latency / cache budgets.
C-7	Commercial / tenancy scope	Retain tier defaults and single-campus pilot; confirm pricing interpretations, cancellation exceptions and future settlement scope.
C-8	Data and recovery policy	Approve account deletion, audit/telemetry retention, backups, RPO/RTO and supported installed-client versions.

Specification alignment
New customer read API names and one-time proof-response recovery are explicit design details for review. This blueprint does not expand the product to payments, a Customer camera scanner, continuous Customer background GPS or a new admin application.

Appendix Sources and design baseline

Project sources

Source	Use in this architecture
Toade3_TRD_v1.1_Mobile_App .pdf	Customer mobile scope, Supabase backend, pricing, fleet limits, auth, live updates, proof, performance targets and proposed review decisions.
architecture.md v1.0.0	Shared app/backend boundaries, role-shaped interfaces, transactions, private data, lifecycle recovery and distribution approach.
Original Toade3 brand asset	Original supplied logo and inherited navy/orange design tokens reused for this PDF. Diagrams are rendered as vector content for sharp scaling.

The separate SRS was not independently supplied; its references are inherited from the TRD. This Customer blueprint should be reconciled with the final SRS, approved contracts and architecture decisions before implementation sign-off.

Official technical references

Reference	Official documentation
Native auth and sessions	Supabase Auth with React Native Session persistence and foreground AppState refresh.
Mobile callbacks	Supabase Native Mobile Deep Linking Callback configuration for mobile identity flows.
Secure local secrets	Expo SecureStore OS-backed storage integration and lifecycle constraints.
Native read caching	Expo SQLite Persistent cache and local database integration.
Native push	Expo Push Notifications overview Push delivery and troubleshooting FAQ
Optional customer location	Expo Location Foreground permission and location integration; Customer baseline needs no rider background task.
iOS distribution	Apple App Review Guidelines Login, notification, privacy and account-lifecycle review gates.

Review and measurement status

Prepared 18 October 2026 as version 1.0.0 for engineering and product review. Recheck compatible SDK versions, store requirements and provider configuration before implementation. Native OS limits, permission denial and push delivery must be tested on physical devices. All performance figures are targets or assumptions until a release test records evidence.

End-to-end completion criterion
A customer can sign in, place one correctly priced order, follow authorized preparation and delivery updates, present a securely held credential, receive server-verified completion and recover safely from network or process interruption.